

# 排队接水

## 问题分析

本题主要目标是在一个水龙头的情况下，通过调整  $n$  个人的排队顺序，使得所有人的平均等待时间最小。题目给出的输入包含总人数  $n$  以及每个人接水所需的时长  $T_i$ ，且保证所有  $T_i$  互不相同。核心任务是计算最小的平均等待时间。由于平均等待时间等于总等待时间除以人数  $n$ ，而  $n$  是固定的，因此问题等价于求总等待时间的最小值。

## 算法策略

为了使总等待时间最小，采用贪心算法的思想。分析排队过程可知，排在队伍第  $i$  位的人接水时，排在他后面的所有人（共  $n - i$  个人）都在等待，而他之前的接水时间对这  $n - i$  个人的等待时间均有贡献。换言之，排在越靠前的人，其接水时间被计入总等待时间的次数越多。具体来说，第 1 个人的接水时间会让后面  $n - 1$  个人等待，第 2 个人的接水时间会让后面  $n - 2$  个人等待，以此类推，最后一名接水的人不会让任何人等待。为了让总加权和最小，我们应当让耗时越短的人排在越靠前的位置，即按照接水时间  $T_i$  从小到大进行排序。

## 代码实现与逻辑

```
#include <iostream>
#include <algorithm>
#include <iomanip>

using namespace std;

int main()
{
    int n;
    cin >> n;

    int times[1005];
    for (int i = 0; i < n; i++) cin >> times[i];

    // 贪心策略：按接水时间升序排序
    sort(times, times + n);

    double total_wait = 0;
    double prefix_sum = 0;
    for (int i = 0; i < n; i++) {
        total_wait += prefix_sum;
        prefix_sum += times[i];
    }

    // 输出平均等待时间，保留1位小数
    cout << fixed << setprecision(1) << total_wait / n << endl;
```

```
    return 0;
}
```

程序核心步骤是调用 `std::sort` 函数对 `times` 数组进行升序排序，保证接水时间短的人优先服务，从而满足贪心选择性质。在计算环节，使用一个循环来累加等待时间。变量 `prefix_sum` 维护的是当前时刻之前的累计接水时间，也就是当前第  $i$  个人开始接水前已经等待的时间；变量 `total_wait` 用于累加所有人的等待时间。在每一次循环中，先将当前的 `prefix_sum` 加到 `total_wait` 中（代表第  $i$  个人的等待时长），然后再将第  $i$  个人的接水时间加到 `prefix_sum` 中，为下一个人做准备。最后，将总等待时间除以  $n$  并格式化输出，保留一位小数。

### 样例验证与复杂度

以样例数据为例，4个人的接水时间分别为 2、3、4、5。排序后顺序不变。第一个人等待时间为 0；第二个人等待第一个人的 2 分钟；第三个人等待前两人的  $2+3=5$  分钟；第四个人等待前三人的  $2+3+4=9$  分钟。总等待时间为  $0 + 2 + 5 + 9 = 16$ 。平均等待时间为  $16/4 = 4.0$ ，与样例输出一致。从时间复杂度来看，主要消耗在于排序操作，复杂度为  $O(n \log n)$ ，对于题目规模来说非常高效。空间复杂度为  $O(n)$ ，用于存储输入数据。综上所述，该解法逻辑严密且能够正确求解最小值。